

RW BLE Android SDK使用说明文档

1. 简介

本文档主要对SDK里所提供的功能接口与使用场景进行解释说明。

此文档仅适用于RW公司的蓝牙设备。

1.1 适用平台与语言

- Android8.0及以上, 语言Kotlin.

1.2 相关术语

- App: 本文指的是手机端或平板电脑上运行的应用程序;
- 设备: 本文指的是可穿戴式硬件设备:如手表,戒指等;
- 上传: 指的是设备向App发送数据;
- 下发: 指的是App向设备发送数据;

1.3 注意事项

1. 使用此SDK时最好结合示例工程 `RW_SDK_DEMO` 使用; 参考例子只需关注NewMainActivity与ScanActivity两个页面代码即可.
2. `DHBluetooth` 大部分指令操作都是通过 订阅相应回调来实现 `DHBluetooth.subscribeData()`; 当不再使用时,请进行取消订阅 `DHBluetooth.dispose()`;

2. 快速开始 (Quick Start)

第1步: 获取最新版本的Android Studio

要想使用 RW BLE SDK for Android 开发项目, 您需要安装Android Studio.

第2步: 手动部署添加依赖库

1. 导入aar到项目build.gradle里

```
implementation files('libs/blesdk_rwfit_release_260130.aar')
```

第3步: 需手机开启蓝牙并请求蓝牙与定位权限

```
private fun getBluePermissions(): List<String> {  
    val permissions = mutableListOf<String>()  
    permissions += Permission.BLUETOOTH_SCAN  
    permissions += Permission.BLUETOOTH_ADVERTISE  
    permissions += Permission.BLUETOOTH_CONNECT  
    permissions += Permission.ACCESS_COARSE_LOCATION  
    permissions += Permission.ACCESS_FINE_LOCATION  
    return permissions  
}
```

第4步: 初始化SDK

```
DHBleSdk.initSDK(this)
```

⚠ Caution

会默认保存蓝牙部分日志文件, 保存在 `Data/appid(com.xxx.xxx)/logger/devices/` 文件夹下. 可 `XLogUtils.setLogEnable(false)` 关闭.

SDK 混淆说明

Release AAR 已对 SDK 内部实现进行混淆, 并已内置用于保留公开 API 的消费者混淆规则。通过 Gradle 正常引入 AAR 时, 这些规则会自动合并到应用的 R8/ProGuard 配置中, 一般不需要额外配置。

如果使用了会丢失 AAR 消费者混淆规则的二次打包方式, 请在应用的 ProGuard 配置中增加以下规则:

```
-keepattributes Signature,*Annotation*,InnerClasses,EnclosingMethod

-keep class com.example.blesdk.DHBleSdk { *; }

-keep class com.example.blesdk.ble.** { *; }
-keep class com.example.blesdk.rwbaselibrary.HandlerCallback { *; }

-keep class com.example.blesdk.bean.** { *; }
-keep class com.example.blesdk.callback.** { *; }

-keep class com.example.blesdk.blering.BaseDataCallback { *; }
-keep class com.example.blesdk.blering.DeviceDataCallback { *; }
-keep class com.example.blesdk.blering.BaseAnswerCallback { *; }
-keep class com.example.blesdk.blering.RingConnectBleCallback { *; }
-keep class com.example.blesdk.blering.RingBleError { *; }

-keep class com.example.blesdk.utils.** { *; }
```

3. 接口说明 (API Reference)

3.1 设备搜索与连接, 绑定与重连

3.1.1 搜索蓝牙

接口说明: 搜索蓝牙设备需使用 `ScanBleService` 先初始化并实现 `ScanDeviceCallback` 回调接口;

```
// 1. 初始化并注册回调
ScanBleService.getService().initBle(this)
ScanBleService.getService().registerScanBleCallback(this)

// 2. 开始搜索
ScanBleService.getService().startScan(true,null)

//3. ScanDeviceCallback接口会回调搜索到的蓝牙设备
public interface ScanDeviceCallback {
    public void onScanDevice(BleDevice device);
    public void onScanFinish();
    public void onError(int errorCode,Exception e);
}
```

```
//4. 取消注册回调
ScanBleService.getService().unRegisterScanBleCallback()
```

3.1.2 停止搜索

接口说明: 停止搜索蓝牙设备

```
ScanBleService.getService().stopScan()
```

3.1.3 连接设备与状态监听

接口说明: 连接指定设备,并监听设备连接状态.

```
// 1. 初始化并注册回调
DHBLESdk.setConnectBleCallback(this)

// 2. 连接设备
DHBLESdk.connectDeviceWithModel(bleDevice)

// 3. 实现并回调蓝牙连接状态
interface RingConnectBleCallback {
    fun onRingConnecting()
    fun onRingConnected()
    fun onRingConnectFailed(reason: RingBleError = RingBleError.UNKNOWN)

    fun onRingDidFunctionMenu(supportMenuBean: SupportMenuBean)
}
```

RingConnectBleCallback 接口说明:

方法	说明
onRingConnecting	连接中
onRingConnected	connectDeviceWithModel后,连接成功会返回.
onRingConnectFailed	蓝牙断开会回调
onRingDidFunctionMenu	成功获取设备配置表后会返回;业务操作应该在此之后操作.

 **Tip**
连接后, 业务操作应该在 onRingDidFunctionMenu 之后才进行操作.

3.1.4 断开连接设备

接口说明: 断开正在连接的设备,并监听设备连接状态

```
DHBLESdk.disconnect()
```

3.1.5 本地绑定与自动重连,解绑

 **Important**

Android本地绑定,自动重连需自行实现, 本SDK不提供此功能;可以连接成功后,保存Mac地址,需要的配置表信息,方便页面显示与重连.

3.1.6 设备功能配置表

Important

因设备型号多, 支持的功能不同,所以引入功能表信息,可查询设备功能支持情况. 具体参考SupportMenuBean类. 可根据业务自行保存功能表内容.

```
fun onRingDidFunctionMenu (supportMenuBean:SupportMenuBean)
```

DeviceFuncV2Model类属性定义:

SupportMenuBean属性	说明
isPushMsgEnableSwitch	是否启用消息控制开关
pushMsgSwitchValue	消息类型支持能力低32位 (bit0-bit31)
pushMsgSwitchValue2	消息类型支持能力高32位 (bit32-bit63) , 旧设备默认为0
activityDataInterval	当天计步明细间隔 (分钟) ; 未配置时按60处理
isAlarm	是否支持闹钟
isBrightScreenSleepTime	是否支持屏幕睡眠时间设置
isBrightScreenTime	是否支持亮屏时长
isNewSport	是否支持多运动;
isRememberSwitch	是否支持Muslim赞念开关
isSupportHrReminder	是否支持HR报警提示功能
isSupportBoReminder	是否支持SP02报警提示功能
isSupportMotoVibrationLevel	是否支持马达震动提醒
isSupportAlarmVibrationDuration	是否支持闹钟震动时长设置
isSupportVibrationInterval	是否支持震动间隔时长设置
isStep	是否支持计步
isHr	是否支持心率
isBloodPress	是否支持血压
isSleep	是否支持睡眠
isBloodOxy	是否支持血氧
isHrv	是否支持心率变异性
isPressure	是否支持压力
isBloodSugar	是否支持血糖

isMuslimCountData	是否支持赞念
isDataTypeTemperature	是否支持体温
isSupportMuslimTimeDisplayMode	是否支持Muslim时间显示模式
isSupportSensorRawPPG	是否支持获取PPG原始数据
isSupportPPGMonitoring	是否支持PPG定时监测
isSupportTemperatureMonitoring	是否支持温度定时监测
isSupportCountReminder	是否支持计数提醒间隔设置
isSupportSensorRawACC	是否支持获取ACC原始数据
isSupportSensorRawPPGRed	是否支持获取PPG Red原始数据
isSupportSensorRawIR	是否支持获取IR红外原始数据
isSupportSensorRawSleep	是否支持睡眠实时数据
isSupportFallDetect	是否支持跌落提醒
isSupportRecording	是否支持录音功能

3.2 设备功能操作

3.2.1 基础功能指令接口

3.2.1.1 Get SDK Version

获取SDK版本号.

方法说明:

```
DHBleSdk.getSDKVersion()
```

调用示例:

```
Log.e("RWSDK", DHBleSdk.getSDKVersion())
```

3.2.1.2 设置用户信息

用户信息设置与计步卡路里,距离有关; 设备初始化时性别为1, 年龄 18, 身高170cm, 体重 65 kg.

订阅 `CommonStatusCallback` 获取结果.

方法说明:

```
fun setUserInfo(personBean: PersonBean)
```

参数说明:

参数	类型	说明	值
personBean	PersonBean	类	gender: 性别 (0.女 1.男) height: 身高cm,浮点型 weight: 体重kg,浮点型 age: 年龄

调用示例:

```
DHBleSdk.subscribeStatus(object : CommonStatusCallback{
    override fun onSuccess(id: Int) {
        Log.e("RWSDK", "user info set ok")
    }

    override fun onFail(id: Int, errorCode: Int) {
        Log.e("RWSDK", "user info set failed")
    }
})

val persionBean = PersonBean()
persionBean.measureUnit = 0
persionBean.gender = 1
persionBean.height = 170.5f
persionBean.weight = 80f
persionBean.age = 20
DHBleSdk.setUserInfo(persionBean)
```

3.2.1.3 获取设备信息

接口说明: 获取固件型号,固件版本号,UI版本号;

订阅 `FirmwareCallback` 获取结果.

```
//1. 订阅LoginDeviceCallback回调
DHBleSdk.subscribeData(object : FirmwareCallback {
    override fun onSuccess() {
    }
    override fun onFail(errorCode: Int) {
        onAppend("ERROR CODE $errorCode")
    }
    override fun onResult(data: FirmVersionBean?) {
        data?.let {
            onAppend("固件版本 --> \n$it")
        }
    }
})

//2. 发送数据, 结果将在FirmwareCallback 里获取到.
DHBleSdk.getFirmwareVersionJL()

//3. 取消订阅
DHBleSdk.dispose(FirmwareCallback)

//FirmVersionBean 实体类
public class FirmVersionBean extends BleSendBean {
    private String deviceClazz = ""; //设备型号
    private String deviceNo = "1.0.0"; //设备版本号
    private int screenType; //0方 1圆
    private int screenWidth; //设备宽
    private int screenHeight; //设备高
```

```
private String uiVersion; //UI版本号
}
```

3.2.1.4 获取电量

接口说明: app获取设备电量

订阅 `PowerCallback` 获取结果.

```
//1. 订阅PowerCallback回调
DHBleSdk.subscribeData(object : PowerCallback {
    override fun onSuccess() {
    }

    override fun onFail(errorCode: Int) {
        onAppend("ERROR CODE $errorCode")
    }

    override fun onResult(data: PowerBean?) {
        data?.let {
            onAppend("设备电量 --> \n$it")
        }
    }
})

//2. 发送数据, 结果将在PowerCallback里获取到.
DHBleSdk.getPowerJL()

//3. 取消订阅
DHBleSdk.dispose(PowerCallback)

//PowerBean 实体类
public class PowerBean implements Parcelable {
    private boolean isLowPower;//低电状态
    private int powerStatus;//充电状态, 0未充电, 1正在充电 2充电完成
    private int power;//电量 0-100
}
```

3.2.1.5 获取与设置视频控制开关

设置是否开启戒指手势控制刷视频; 此功能需配对蓝牙HID. HID配对可使用 `BlueToothUtils createOrRemoveBond` (type 1 匹配 2 取消匹配) 进行配对HID, 也可自行实现.

订阅 `videoHidCallback` 获取结果.

方法说明:

```
fun setVideoHidJL(videoHidBean: VideoHidBean)
```

参数说明:

参数	类型	说明	值
videoHidBean	VideoHidBean	类	hidOpen: 0.关闭 1.视频打开 2. boook 3. music

调用示例:

```
//设置视频控制开关
DHBLESdk.subscribeData(videoHidCallback)
val videoHidBean = VideoHidBean()
videoHidBean.hidOpen = 1 //Whether to open short video control
DHBLESdk.setVideoHidJL(videoHidBean)

// 获取视频控制开关
DHBLESdk.subscribeData(videoHidCallback)
DHBLESdk.getVideoHidJL()
```

3.2.1.6 获取与设置LED亮屏强度

配置表属性: isLEDLight ;
订阅 BrightLedLevelCallback 获取结果.

方法说明:

```
fun setRingLedLevel(brightScreenBean: BrightScreenLedBean)
```

参数说明:

参数	类型	说明	值
brightScreenBean	BrightScreenLedBean	类	isOpen: false为off,ture为(1-3Level) lcdLevel: 1微光, 2柔光, 3强光

调用示例:

```
// 获取LED亮屏强度
DHBLESdk.subscribeData(brightLedLevelCallback)
DHBLESdk.getRingLedLevel()

//设置LED亮屏强度
DHBLESdk.subscribeData(brightLedLevelCallback)
val tBrightScreenLedBean = BrightScreenLedBean()
tBrightScreenLedBean.isOpen = true //false为off,ture为(1-3Level)
tBrightScreenLedBean.lcdLevel = 3 //1-3Level: 1 low 2 mid 3 high
DHBLESdk.setRingLedLevel(tBrightScreenLedBean)
```

3.2.1.7 获取与设置佩戴位置

配置表属性: isWearDir ;
订阅 WearHandCallback 获取结果.

```
// 获取佩戴位置
DHBLESdk.subscribeData(ringWearHandCallback)
DHBLESdk.getRingWearDir()

//设置佩戴位置
DHBLESdk.subscribeData(ringWearHandCallback)
DHBLESdk.setRingWearHand(false) //False is left hand, true is right hand
```


3.2.1.8 启动与关闭拍照

启动拍照功能后,设备可通过手势控制app自定义相机拍照

订阅 `TakePhotoCallback` 设备发出拍照通知,进行拍照.

配置表属性: `isTakePhoto` ;

```
//APP进相机界面启动 1为控制设备进对应界面, 0为控制设备退出
DHBleSdk.subscribeData(takePhotoCallback)
DHBleSdk.controlTakePhotoJL(1) //Open Photo打开拍照

//0为控制设备退出
DHBleSdk.dispose(takePhotoCallback)
DHBleSdk.controlTakePhotoJL(0) //Close photo taking关闭拍照

//监听设备发出拍照指令
/**
 * Camera control monitoring 拍照控制监听
 */
private val takePhotoCallback by lazy {
    object : TakePhotoCallback {
        override fun onSuccess() {
            Log.e("RWSDK", "Output: TakePhotoCallback onSuccess")
        }

        override fun onFail(errorCode: Int) {

        }

        override fun onResult(data: Int?) {
            Log.e("RWSDK", "Output: TakePhotoCallback onResult " + data)
            data.let {
                when (it){
                    2 -> {
                        //TODO Start the phone's custom camera to take photos 启动手机自定义相机拍照
                    }
                }
            }
        }
    }
}
```

3.2.1.9 查找设备

调用查找后设备灯或屏幕会亮.

方法说明:

```
fun controlFindDeviceJL()
```

调用示例:

```
DHBleSdk.controlFindDeviceJL()
```

3.2.1.10 关机,恢复出厂设置

订阅 `DeviceControlCallback` 获取结果.

方法说明:

```
fun setPowerOffJL(type: Int)
```

参数说明:

参数	类型	说明	值
type	Int	整形	关机: Constants.CONTROL_DEVICE_POWER_OFF 恢复出厂: Constants.CONTROL_DEVICE_RECOVERY

调用示例:

```
// 关机
DHBleSdk.setPowerOffJL(Constants.CONTROL_DEVICE_POWER_OFF) //Shutdown (关机)

// 恢复出厂
DHBleSdk.setPowerOffJL(Constants.CONTROL_DEVICE_RECOVERY) //Factory Reset(恢复出厂)

//可订阅subscribe DeviceControlCallback回调
```

3.2.1.11 闹钟

3.2.1.11.1 获取已设置闹钟

配置表属性: `isAlarm`

订阅 `AlarmCallback` 回调;

方法说明:

```
DHBleSdk.getAlarmRemindJL();
```

调用示例:

```
//获取设备里已保存的闹钟
DHBleSdk.subscribeData(alarmCallback)
DHBleSdk.getAlarmRemindJL()
```

3.2.1.11.2 设置闹钟

当前协议不支持单独修改闹钟，任何单个闹钟的开关或删除操作，均需重新下发全部闹钟配置。

订阅 `AlarmCallback` 回调;

方法说明:

```
fun setAlarmRemindJL(reminderBeans: List<AlarmRemainderBean>)
```

参数说明:

参数	类型	说明	值
reminderBeans	List	闹钟数组	isOpen: true开/false关 repeatModel: IntArray(7)周日至周六,要重复的对应置1 startHour: 闹钟开始时 startMin: 闹钟开始分 alarmTag: 设置为空字符串;

调用示例:

```
//设置闹钟
DHBleSdk.subscribeData(alarmCallback)

val params = mutableListOf<AlarmRemainderBean>()
val alarmRemainderBean = AlarmRemainderBean()
alarmRemainderBean.alarmTag = ""
alarmRemainderBean.repeatModel = IntArray(7) //单次; 周日至周六,要重复的对应置1
alarmRemainderBean.startHour = 7
alarmRemainderBean.startMin = 0
alarmRemainderBean.isOpen = true
alarmRemainderBean.alarmId = 0
params += alarmRemainderBean

val alarmRemainderBean2 = AlarmRemainderBean()
alarmRemainderBean2.alarmTag = ""
alarmRemainderBean2.repeatModel = IntArray(7) //单次; 周日至周六,要重复的对应置1
alarmRemainderBean2.startHour = 8
alarmRemainderBean2.startMin = 0
alarmRemainderBean2.isOpen = false
alarmRemainderBean2.alarmId = 0
params += alarmRemainderBean2

DHBleSdk.setAlarmRemindJL(params)
```

3.2.1.11.3 删除所有闹钟

```
DHBleSdk.deleteAllAlarmRemindJL;

订阅 AlarmCallback 回调;
```

方法说明:

```
DHBleSdk.deleteAllAlarmRemindJL
```

```
//订阅回调
DHBleSdk.subscribeData(alarmCallback)
//删除所有闹钟
DHBleSdk.deleteAllAlarmRemindJL()
```

3.2.1.12 震动次数设置与获取

```
配置表属性: isSupportMotoVibrationLevel

设备震动次数;

订阅 VibrationCountCallback 回调.
```

方法说明:

```
fun setVibrationCount(level:Int, count: Int)

fun getVibrationCount()
```

参数说明:

参数	类型	说明	值
level	Int	整形	震动强度, 0:关闭 1:低 2: 中 3: 高; 未定义此功能的可忽略
count	Int	整形	震动次数可以被设置(0-6次), 初始默认2次.设置0次不震动

调用示例:

```
//设置
DHBleSdk.subscribeData(vibrationCountCallback)
DHBleSdk.setVibrationCount(1, 2) //震动次数2次

//获取
DHBleSdk.subscribeData(vibrationCountCallback)
DHBleSdk.getVibrationCount()
```

3.2.1.13 屏幕睡眠模式设置与获取

设置屏幕睡眠开启与时间;
配置表属性: isBrightScreenSleepTime
订阅 BrightTimeCallback 获取结果.

方法说明:

```
fun setRingBrightScreenSleepTime(briScreenTime: BrightScreenTimeBean)

fun getRingBrightScreenSleepTime()
```

参数说明:

参数	类型	说明	值
BrightScreenTimeBean	类		isOpen, 开关打开YES或关闭NO startHour, 开始时间小时 startMin,开始时间分钟 endHour,结束时间小时 endMin,结束时间分钟

调用示例:

```
//订阅
DHBleSdk.subscribeData(brightTimeCallback)

//设置
val briScreenTime = BrightScreenTimeBean()
briScreenTime.isOpen = true
briScreenTime.startHour = 20 //晚上8点至早上8点睡眠
```

```
briScreenTime.startMin = 0
briScreenTime.endHour = 8
briScreenTime.endMin = 0

DHBleSdk.setRingBrightScreenSleepTime(briScreenTime)

//获取
DHBleSdk.getRingBrightScreenSleepTime()
```

3.2.1.14 消息与来电

3.2.1.14.1 消息推送

APP主动向设备推送消息通知(非ANCS), 消息开关由APP自行控制.

方法说明:

```
fun setPushMsgJL(msgPushBean: MsgPushBean)
```

参数说明:

参数	类型	说明	值
MsgPushBean	类		见MsgPushBean类定义

调用示例:

```
val messageBean = MsgPushBean()
messageBean.appId = "com.ten.wenxin"
messageBean.title = "1111"
messageBean.content = "8888"
DHBleSdk.setPushMsgJL(messageBean)
```

3.2.1.14.2 来电控制

设备端触发接听或挂断来电时, APP需监听设备指令并执行对应电话操作.

方法说明:

```
fun controlPhoneJL(controlType: Int)
```

参数说明:

参数	类型	说明	值
controlType	Int	来电控制类型	0: 接听 1: 挂断

调用示例:

```
// 接听来电
DHBleSdk.controlPhoneJL(0)

// 挂断来电
DHBleSdk.controlPhoneJL(1)
```

3.2.1.14.3 音乐控制

设备端触发音乐控制(播放/暂停/上一曲/下一曲等)时, APP需监听设备指令并执行对应操作.

订阅 `MusicPushSettingCallback` 接收设备音乐控制事件.

MusicPushSettingCallback 返回值说明:

值	说明
1	播放
2	暂停
3	上一曲
4	下一曲
5	音量加
6	音量减

调用示例:

```
DHBluetooth.subscribeData(object : MusicPushSettingCallback {
    override fun onResult(data: Int?) {
        when (data) {
            1 -> { /* 播放 */ }
            2 -> { /* 暂停 */ }
            3 -> { /* 上一曲 */ }
            4 -> { /* 下一曲 */ }
            5 -> { /* 音量加 */ }
            6 -> { /* 音量减 */ }
        }
    }
    override fun onFail(errorCode: Int) {}
    override fun onSuccess() {}
})
```

3.2.1.15 获取与设置赞念是否打开

设置赞念功能是否打开;

配置表属性: `isRememberSwitch`

订阅 `MuslimCountSwitchCallback` 获取结果.

方法说明:

```
fun deviceRememberSwitch(status: Int)
fun deviceRememberSwitchGet()
```

参数说明:

参数	类型	说明	值
status	Int		0: 关闭 1: 打开

调用示例:

```
//设置
DHBSdk.subscribeData(muslimCountSwitchCallback)
DHBSdk.deviceRememberSwitch(1)

//获取
DHBSdk.deviceRememberSwitchGet()
```

3.2.1.16 获取与设置心率/血氧报警配置

设置心率与血氧通知报警数据功能; 报警提示会通过 HrBoActualReminderCallback 实时通知出来.

配置表属性: isSupportHrReminder

订阅 HrReminderCallback , BoReminderCallback 获取结果.

方法说明:

```
fun deviceGetHrAlertCmd()

fun deviceSetHrAlertCmd(status: Int, value: Int, underValue:Int)

fun deviceGetBoAlertCmd()

fun deviceSetBoAlertCmd(status: Int, value: Int)
```

参数说明:

参数	类型	说明	值
status	Int		1: 开, 0: 关; overValue: 报警值, 默认值为 心率超过160, 血氧低于94%,
value	Int		超出报警值, 默认值为 心率超过160
underValue	Int		低于设置值报警; 如果获取到为0xff,代表不支持此项功能;

注意: 通过 deviceGetHrAlertCmd()获取如果underValue为0xff,代表不支持此项功能.

实时报警返回值HrBoActualReminderBean说明:

HrBoActualReminderBean参数	类型	说明	值
type	Int		0: 心率超出报警, 1: 血氧报警; 2: 心率低于报警
remindValue	Int		报警值

调用示例:

```
//设置
DHBSdk.subscribeData(hrReminderCallback)
DHBSdk.deviceSetHrAlertCmd(1, 140, 0xff)
```

```

//获取
DHBleSdk.subscribeData(hrReminderCallback)
DHBleSdk.deviceGetHrAlertCmd()

//报警结果通知推送
DHBleSdk.subscribeData(hrBoActualReminderCallback) //Alert Message

private val hrBoActualReminderCallback by lazy {
    object : HrBoActualReminderCallback{
        override fun onResult(data: HrBoActualReminderBean) {
            Log.e("RWSDK", "output: HrBoActualReminderCallback data " + data.type + " value " +
data.remindValue)
            if (data.type == 0){ // HR Over Value Alarm

            }
            else if (data.type == 1){// SP02 Over Value Alarm

            }
            else if (data.type == 2){// HR Under Value Alarm

            }
        }
    }

    override fun onFail(errorCode: Int) {
        Log.e("RWSDK", "output: HrBoActualReminderCallback errorCode " + errorCode)
    }

    override fun onSuccess() {
        Log.e("RWSDK", "output: HrBoActualReminderCallback onSuccess")
    }
}
}

```

3.2.1.17 获取与设置亮屏时长

配置表属性: `isBrightScreenTime`

订阅 `BrightTimeCallback` 获取结果.

方法说明:

```
fun getBrightScreenTimeJL()
```

```
fun setBrightScreenTimeJL(briScreenTime: BrightScreenTimeBean)
```

参数说明:

BrightScreenTimeBean参数	类型	说明	值
timeSecond	Int	亮屏时长,秒(s), 范围0-30s;	
durationNums	String	设备支持的时长值, 获取到如果有值,以逗号隔开;	

调用示例:


```
//设置
val briScreenTime = BrightScreenTimeBean()
briScreenTime.timeSecond = 10 //亮屏10s
DHBleSdk.subscribeData(brightTimeCallback)
DHBleSdk.setBrightScreenTimeJL(briScreenTime)

//获取
DHBleSdk.getBrightScreenTimeJL()
```

3.2.1.18 获取与设置抬腕亮屏时长

配置表属性: isRaiseBrightScreen
订阅 BrightCallback 获取结果.

方法说明:

```
fun getRaiseBrightScreenJL()

fun setRaiseBrightScreenJL(brightScreenBean: BrightScreenBean)
```

参数说明:

BrightScreenBean参数	类型	说明	值
isOpen	Int	true:开启; false: 关闭	
startHour	Int	开始时间小时	
startMin	Int	开始时间分钟	
endHour	Int	结束时间小时	
endMin	Int	结束时间分钟	

调用示例:

```
//设置
val rasieScreenTime = BrightScreenBean()
rasieScreenTime.isOpen = true
rasieScreenTime.startHour = 8 //早上8点至晚上8点 抬腕亮屏
rasieScreenTime.startMin = 0
rasieScreenTime.endHour = 20
rasieScreenTime.endMin = 0
DHBleSdk.subscribeData(raiseBrightTimeCallback)
DHBleSdk.setRaiseBrightScreenJL(rasieScreenTime)

//获取
DHBleSdk.getRaiseBrightScreenJL()
```

3.2.1.19 设置时间格式12/24小时制

带屏显示时间的设备才有效。

订阅 `CommonStatusCallback` 获取结果。

方法说明:

```
fun ringSetTimeformat(type: Int)
```

参数说明:

参数	类型	说明	
timeformat	Int	0: 24小时制 1: 12小时制	

调用示例:

```
DHBleSdk.subscribeStatus(object : CommonStatusCallback{
    override fun onSuccess(id: Int) {
        Log.e("RWSDK", "time format set ok")
    }

    override fun onFail(id: Int, errorCode: Int) {
        Log.e("RWSDK", "time format set failed")
    }
})
DHBleSdk.ringSetTimeformat(0)
```

3.2.1.20 闹钟震动时长设置与获取

设置闹钟震动次数;

配置表属性: `isSupportAlarmVibrationDuration`

订阅 `AlarmVibrationDurationCallback` 获取结果。

方法说明:

```
fun setAlarmVibrationDuration(count: Int)

fun getAlarmVibrationDuration()
```

参数说明:

参数	类型	说明	值
count	Int	整形	震动次数(0-6), 默认2次, 设置0次为不震动

调用示例:

```
//设置
DHBLESdk.subscribeData(alarmVibrationDurationCallback)
DHBLESdk.setAlarmVibrationDuration(2) //2次

//获取
DHBLESdk.subscribeData(alarmVibrationDurationCallback)
DHBLESdk.getAlarmVibrationDuration()
```

3.2.1.21 触摸事件通知

设备触摸事件通知, 设备主动上报. 触摸操作无论熄屏与否都会上报, 由APP定义响应行为.

订阅 `TouchEventCallback` 接收触摸事件.

`TouchEventCallback` 返回 `int[]` 数据说明:

索引	说明	值
[0]	按键类型	1: 触摸按键(默认), 2: 跌落(需开启跌落提醒3.2.1.24)
[1]	触摸类型	1: 单击, 2: 双击, 3: 三击, 4: 长按, 5: 甩动. 按键类型为2(跌落)时, 触摸类型默认为1

调用示例:

```
//在onCreate中订阅
DHBLESdk.subscribeData(touchEventCallback)

private val touchEventCallback by lazy {
    object : TouchEventCallback {
        override fun onResult(data: IntArray?) {
            data?.let {
                val keyType = it[0] // 1:触摸按键
                val touchType = it[1] // 1:单击 2:双击 3:三击 4:长按 5:甩动
                Log.e("RWSDK", "TouchEvent keyType=$keyType touchType=$touchType")
            }
        }
        override fun onFail(errorCode: Int) {}
        override fun onSuccess() {}
    }
}
```

3.2.1.22 震动间隔时长设置与获取

设置每次震动之间的间隔时间, 用于调节震动节奏;

配置表属性: `isSupportVibrationInterval`

订阅 `VibrationIntervalCallback` 获取结果.

方法说明:

```
fun setVibrationInterval(intervalMs: Int)

fun getVibrationInterval()
```

参数说明:

参数	类型	说明	值
intervalMs	Int	整形	间隔时长(100-1000ms), 默认500ms

调用示例:

```
//设置
DHBLESdk.subscribeData(vibrationIntervalCallback)
DHBLESdk.setVibrationInterval(500) //500ms

//获取
DHBLESdk.subscribeData(vibrationIntervalCallback)
DHBLESdk.getVibrationInterval()
```

3.2.1.23 心率校正(工厂测试)

启动设备心率校正模式. 发送校正指令后, 设备会返回两条数据:
第1条 result=0 表示校正中; 第2条 result非0 表示校正完成.
订阅 `FactoryTestCallback` 获取返回结果, `onResult` 返回 `long[]`: `[0]=testMode`, `[1]=result`.

方法说明:

```
fun startFactoryTest(testMode: Int)
```

参数说明:

参数	类型	说明	值
testMode	Int	测试模式	0x15: 心率校正

调用示例:

```
DHBLESdk.subscribeData(factoryTestCallback)
DHBLESdk.startFactoryTest(0x15)

private val factoryTestCallback by lazy {
    object : FactoryTestCallback {
        override fun onResult(data: LongArray?) {
            data?.let {
                if (it[1] == 0L) {
                    Log.e("RWSDK", "HR calibrating...")
                } else {
                    Log.e("RWSDK", "HR calibration done, result=${it[1]}")
                }
            }
        }
        override fun onFail(errorCode: Int) {}
        override fun onSuccess() {}
    }
}
```

3.2.1.24 跌落提醒设置

设置或获取跌落提醒开关. 开启后设备检测到跌落时会通过触摸事件通知(3.2.1.21)上报.

跌落事件通过 `TouchEventCallback` 返回, 按键类型(keyType)=2 表示跌落事件.

配置表属性: `isSupportFallDetect`

订阅 `FallDetectCallback` 获取设置/获取结果.

方法说明:

```
fun setFallDetect(enable: Boolean)

fun getFallDetect()
```

参数说明:

参数	类型	说明	值
enable	Boolean	开关	true: 开, false: 关

调用示例:

```
//获取跌落提醒开关
DHBleSdk.subscribeData(fallDetectCallback)
DHBleSdk.getFallDetect()

//设置跌落提醒开启
DHBleSdk.subscribeData(fallDetectCallback)
DHBleSdk.setFallDetect(true)

private val fallDetectCallback by lazy {
    object : FallDetectCallback {
        override fun onResult(data: Int?) {
            Log.e("RWSdk", "FallDetect state: $data (0=off, 1=on)")
        }
        override fun onFail(errorCode: Int) {}
        override fun onSuccess() {}
    }
}
```

3.2.1.25 计数提醒间隔设置

设置或获取计数提醒间隔. 开启后用户完成一次计数操作开始计时, 达到设定间隔后设备震动一次提醒继续计数.

配置表属性: `isSupportCountReminder`

订阅 `CountReminderIntervalCallback` 获取结果.

方法说明:

```
fun setCountReminderInterval(intervalMinutes: Int)

fun getCountReminderInterval()
```

参数说明:

参数	类型	说明	值
intervalMinutes	Int	间隔分钟	0: 关闭, 30/60/90/120: 提醒间隔(分钟)

调用示例:

```
//获取计数提醒间隔
DHBLESdk.subscribeData(countReminderCallback)
DHBLESdk.getCountReminderInterval()

//设置计数提醒间隔60分钟
DHBLESdk.subscribeData(countReminderCallback)
DHBLESdk.setCountReminderInterval(60)

//关闭计数提醒
DHBLESdk.subscribeData(countReminderCallback)
DHBLESdk.setCountReminderInterval(0)

private val countReminderCallback by lazy {
    object : CountReminderIntervalCallback {
        override fun onResult(data: Int?) {
            Log.e("RWSDK", "CountReminderInterval: $data min (0=off)")
        }
        override fun onFail(errorCode: Int) {}
        override fun onSuccess() {}
    }
}
```

3.2.2 健康数据同步(实时单次与全天检测)

健康数据检测有两种方式: 实时单次检测与全天检测。健康数据包括心率,血氧,压力,HRV,睡眠等, 睡眠无实时检测。

(1) 实时单次检测: APP侧启动设备进入单次检测,检测完后马上返回结果。

(2) 全天检测: 可设置间隔时间,比如30分钟或60分钟设备会进行检测并保存值; **app**一直不同步情况下,设备可保存3-6天的数据。

3.2.2.1 实时检测-启动与关闭设备健康数据检测

启动健康数据检测(心率,血氧,HRV,压力,血糖等);

订阅 `HealthDataBroCallback` 测试完成设备会通知app;

订阅 `HealthDataControlCallback` 测试中实时值设备会通知app;

⚠ Caution

同一时间只能开启一种健康检测类型, 必须等当前检测结束(收到完成回调)或主动关闭后, 才能启动新的检测类型. 同时开启多种会导致检测异常.

方法说明:

```
fun controlHealthDataJL(healthType: Byte, testStatus: Byte)
```

参数说明:

参数	类型	说明	值
healthType	Byte	健康数据类型	心率: CmdConstants.JL_HR_DATA_TRANSFER_KEY 血氧: CmdConstants.JL_BO_DATA_TRANSFER_KEY HRV: CmdConstants.JL_HRV_DATA_TRANSFER_KEY 压力: CmdConstants.JL_PRESSURE_DATA_TRANSFER_KEY 血糖: CmdConstants.JL_BLOODSUGAR_DATA_TRANSFER_KEY 血压: CmdConstants.JL_BP_DATA_TRANSFER_KEY 体温: CmdConstants.JL_TEMP_DATA_TRANSFER_KEY
testStatus	Byte	启动/关闭	启动: 1 关闭: 0

调用示例:

```
//启动心率测试
DHBleSdk.subscribeData(healthDataBroCallback) //Monitor real-time health data return (监听实时健康数据返回)
DHBleSdk.subscribeData(testHrCallback) //Monitor control command results (监听控制指令结果)
DHBleSdk.controlHealthDataJL(CmdConstants.JL_HR_DATA_TRANSFER_KEY, 1)

//关闭心率测试
DHBleSdk.subscribeData(testHrCallback)
DHBleSdk.controlHealthDataJL(CmdConstants.JL_HR_DATA_TRANSFER_KEY, 0)

// 监听测量中实时数值改变
private val healthDataBroCallback by lazy {
    object : HealthDataBroCallback{
        override fun onResult(data: HealthDataSyncBean?) {
            data?.let {
                when (it.dataType) {
                    Constants.RingHealthType.HR -> { //Heart Rate
                        Log.e("RWSDK", "Output: hr Value " + it.hrPartData.last().hr)
                    }
                    Constants.RingHealthType.HRV -> { //HRV
                        Log.e("RWSDK", "Output: HRV Value " + it.hrPartData.last().hr)
                    }
                    Constants.RingHealthType.BLOOD_OXY -> { //Blood Oxygen(血氧)
                        Log.e("RWSDK", "Output: Blood Oxygen Value " + it.boPartData.last().bo)
                    }
                    Constants.RingHealthType.PRESSURE -> { //压力 Stress
                        Log.e("RWSDK", "Output: Stress Value " + it.pressurePartData.last().pressure)
                    }
                    Constants.RingHealthType.BLOOD_SUGAR -> { //血糖 BloodSugar
                        Log.e("RWSDK", "Output: BloodSugar Value " + it.tempPartData.last().temp)
                    }
                    Constants.RingHealthType.MUSLIM_COUNT -> { //Msulim Count 赞念
                        Log.e("RWSDK", "赞念 Value " + it.muslimCountPartData.count)
                    }
                    Constants.RingHealthType.BLOOD_BP -> { //血压 Blood Pressure
                        Log.e("RWSDK", "Blood Pressure Value " + it.bpPartData.last().dp + " "
+it.bpPartData.last().sp)
                    }
                    Constants.RingHealthType.TEMPERATURE -> { //体温 Temperature
                        Log.e("RWSDK", "Temperature Value " + it.tempPartData.last().temp / 10.0)
                    }
                }
            }
        }
    }
}
```

```

        }
    }
}
}
override fun onFail(errorCode: Int) {

}

override fun onSuccess() {

}

}
}

//监听测试完成结果
private val testHrCallback by lazy {
    object : HealthDataControlCallback {
        override fun onSuccess() {
            Log.e("RWSDK", "Output: HealthDataControlCallback Control onSuccess")
        }

        override fun onResult(data: Int?) {
            Log.e("RWSDK", "Output: HealthDataControlCallback onResult " + data)
            data?.let {
                if (data >= 10){
                    Log.e("RWSDK", "Output: Measurement completed (测量完成)")
                }
            }
        }

        override fun onFail(errorCode: Int) {

        }
    }
}

```

3.2.2.2 全天检测-设置健康数据全天监听间隔

设置健康数据(心率,血氧,HRV,压力,血糖)全天监听间隔, 单位分钟.

注意事项:暂间隔只有心率可设置30分钟与60分钟, 其它(血氧,HRV,压力,血糖)只能设置开与关; 开始与结束时间固定全天,不可修改.

3.2.2.2.1 心率检测设置与获取

间隔只有心率可设置30分钟与60分钟; 订阅回调: `TimedHeartRateCallback`;

方法说明:

```
fun setTimedHeartRateJL(reminderBean: DrinkReminderBean)
```

```
fun getTimedHeartRateJL()
```

参数说明:

参数	类型	说明	值
reminderBean	DrinkReminderBean	类	isOpen: true开/false关 remindDuration: 间隔时间30或60分钟 startHour: 0 固定0不能修改 startMin: 0 固定0不能修改 endHour: 23 固定23不能修改 endMin: 59 固定59不能修改;

调用示例:

```
// 1. Set HeartRate Monitor(设置心率监听)
DHBleSdk.subscribeData(hrMonitorCallback)
val hrMonitorBean = DrinkReminderBean()
hrMonitorBean.isOpen = true //Heart rate monitoring switch
hrMonitorBean.remindDuration = 60 //Heart rate monitoring interval unit is minutes, only 30 minutes
and 60 minutes
hrMonitorBean.startHour = 0 //fixed
hrMonitorBean.startMin = 0 //fixed
hrMonitorBean.endHour = 23 //fixed
hrMonitorBean.endMin = 59 //fixed
DHBleSdk.setTimedHeartRateJL(hrMonitorBean)

//1. 获取心率监听
DHBleSdk.subscribeData(hrMonitorCallback)
DHBleSdk.getTimedHeartRateJL()
```

3.2.2.2.2 血氧检测设置与获取

间隔血氧只可设置60分钟; 订阅回调: `TimedBloodOxygenCallback`;

配置表属性: `isBloodOxy`;

方法说明:

```
fun setTimedBloodOxygenJL(reminderBean: DrinkReminderBean)

fun getTimedBloodOxygenJL()
```

参数说明:

参数	类型	说明	值
reminderBean	DrinkReminderBean	类	isOpen: true开/false关 remindDuration: 间隔时间固定60分钟 startHour: 0 固定0不能修改 startMin: 0 固定0不能修改 endHour: 23 固定23不能修改 endMin: 59 固定59不能修改;

调用示例:

```
// 2. Set Blood oxygen Monitor(设置血氧监听)
DHBleSdk.subscribeData(timedBloodOxygenCallback)
val healthMonitorBean = DrinkReminderBean()
healthMonitorBean.isOpen = true //BloodOxygen monitoring switch
healthMonitorBean.remindDuration = 60 //fixed 60 minutes
```

```
healthMonitorBean.startHour = 0 //fixed
healthMonitorBean.startMin = 0 //fixed
healthMonitorBean.endHour = 23 //fixed
healthMonitorBean.endMin = 59 //fixed
DHBleSdk.setTimedBloodOxygenJL(healthMonitorBean)

//2. 获取血氧监听
DHBleSdk.subscribeData(timedBloodOxygenCallback)
DHBleSdk.getTimedBloodOxygenJL()
```

3.2.2.2.3 心率变异性(HRV)检测设置与获取

间隔HRV只可设置60分钟; 订阅回调: `TimedHrvCallback`;

配置表属性: `isHrv` ;

方法说明:

```
fun setTimedHRVJL(reminderBean: DrinkReminderBean)

fun getTimedHRVJL()
```

参数说明:

参数	类型	说明	值
reminderBean	DrinkReminderBean	类	isOpen: true开/false关 remindDuration: 间隔时间固定60分钟 startHour: 0 固定0不能修改 startMin: 0 固定0不能修改 endHour: 23 固定23不能修改 endMin: 59 固定59不能修改;

调用示例:

```
// 3. Set HRV Monitor(设置HRV监听)
DHBleSdk.subscribeData(hrvDataCallback)
val healthMonitorBean = DrinkReminderBean()
healthMonitorBean.isOpen = true
healthMonitorBean.remindDuration = 60 //fixed 60 minutes
healthMonitorBean.startHour = 0 //fixed
healthMonitorBean.startMin = 0 //fixed
healthMonitorBean.endHour = 23 //fixed
healthMonitorBean.endMin = 59 //fixed
DHBleSdk.setTimedHRVJL(healthMonitorBean)

//3. 获取HRV监听
DHBleSdk.subscribeData(hrvDataCallback)
DHBleSdk.getTimedHRVJL()
```

3.2.2.2.4 压力检测设置与获取

间隔压力只可设置60分钟; 订阅回调: `TimedStressCallback`;

配置表属性: `isPressure` ;

方法说明:

```
fun setTimedStressJL(reminderBean: DrinkReminderBean)

fun getTimedStressJL()
```

参数说明:

参数	类型	说明	值
reminderBean	DrinkReminderBean	类	isOpen: true开/false关 remindDuration: 间隔时间固定60分钟 startHour: 0 固定0不能修改 startMin: 0 固定0不能修改 endHour: 23 固定23不能修改 endMin: 59 固定59不能修改;

调用示例:

```
// 4. 设置压力监听
DHBleSdk.subscribeData(stressDataCallback)
val healthMonitorBean = DrinkReminderBean()
healthMonitorBean.isOpen = true //Stress monitoring switch
healthMonitorBean.remindDuration = 60 //fixed 60 minutes
healthMonitorBean.startHour = 0 //fixed
healthMonitorBean.startMin = 0 //fixed
healthMonitorBean.endHour = 23 //fixed
healthMonitorBean.endMin = 59 //fixed
DHBleSdk.setTimedStressJL(healthMonitorBean)

//4. 获取压力监听
DHBleSdk.subscribeData(stressDataCallback)
DHBleSdk.getTimedStressJL()
```

3.2.2.2.5 血糖检测设置与获取

间隔血糖只可设置60分钟; 订阅回调: `TimedBloodSugarCallback`;
配置表属性: `isBloodSugar` ;

方法说明:

```
fun setTimedBloodSugarJL(reminderBean: DrinkReminderBean)

fun getTimedBloodSugarJL()
```

参数说明:

参数	类型	说明	值
reminderBean	DrinkReminderBean	类	isOpen: true开/false关 remindDuration: 间隔时间固定60分钟 startHour: 0 固定0不能修改 startMin: 0 固定0不能修改 endHour: 23 固定23不能修改 endMin: 59 固定59不能修改;

调用示例:

```
// 5. 设置血糖监听
DHBLESdk.subscribeData(bloodSugarDataCallback)
val healthMonitorBean = DrinkReminderBean()
healthMonitorBean.isOpen = true //BloodSugar monitoring switch
healthMonitorBean.remindDuration = 60 //fixed 60 minutes
healthMonitorBean.startHour = 0 //fixed
healthMonitorBean.startMin = 0 //fixed
healthMonitorBean.endHour = 23 //fixed
healthMonitorBean.endMin = 59 //fixed
DHBLESdk.setTimedBloodSugarJL(healthMonitorBean)

//5. 获取血糖监听
DHBLESdk.subscribeData(bloodSugarDataCallback)
DHBLESdk.getTimedBloodSugarJL()
```

3.2.2.2.6 血压检测设置与获取

间隔血压只可设置60分钟; 订阅回调: `TimedBloodPressureCallback`;

配置表属性: `isBloodPress` ;

方法说明:

```
fun setTimedBloodPressureJL(reminderBean: DrinkReminderBean)

fun getTimedBloodPressureJL()
```

参数说明:

参数	类型	说明	值
reminderBean	DrinkReminderBean	类	isOpen: true开/false关 remindDuration: 间隔时间固定60分钟 startHour: 0 固定0不能修改 startMin: 0 固定0不能修改 endHour: 23 固定23不能修改 endMin: 59 固定59不能修改;

调用示例:

```
// 6. 设置血压监听
DHBLESdk.subscribeData(timedBloodPressureCallback)
val healthMonitorBean = DrinkReminderBean()
healthMonitorBean.isOpen = true //BloodPressure monitoring switch
healthMonitorBean.remindDuration = 60 //fixed 60 minutes
healthMonitorBean.startHour = 0 //fixed
healthMonitorBean.startMin = 0 //fixed
healthMonitorBean.endHour = 23 //fixed
healthMonitorBean.endMin = 59 //fixed
DHBLESdk.setTimedBloodPressureJL(healthMonitorBean)

//6. 获取血压监听
DHBLESdk.subscribeData(timedBloodPressureCallback)
DHBLESdk.getTimedBloodPressureJL()
```

3.2.2.2.7 体温检测设置与获取

间隔体温可设置30分钟与60分钟; 订阅回调: `TimedBodyTemperatureCallback`;

配置表属性: `isSupportTemperatureMonitoring`

方法说明:

```
fun setTimedBodyTemperature(reminderBean: DrinkReminderBean)
```

```
fun getTimedBodyTemperature()
```

参数说明:

参数	类型	说明	值
reminderBean	DrinkReminderBean	类	isOpen: true开/false关 remindDuration: 间隔时间30或60分钟 startHour: 0 固定0不能修改 startMin: 0 固定0不能修改 endHour: 23 固定23不能修改 endMin: 59 固定59不能修改;

调用示例:

```
// 7. 设置体温监听
DHBleSdk.subscribeData(timedBodyTemperatureCallback)
val healthMonitorBean = DrinkReminderBean()
healthMonitorBean.isOpen = true
healthMonitorBean.remindDuration = 60
healthMonitorBean.startHour = 0
healthMonitorBean.startMin = 0
healthMonitorBean.endHour = 23
healthMonitorBean.endMin = 59
DHBleSdk.setTimedBodyTemperature(healthMonitorBean)

//7. 获取体温监听
DHBleSdk.subscribeData(timedBodyTemperatureCallback)
DHBleSdk.getTimedBodyTemperature()
```

3.2.2.3 全天检测-同步健康历史数据

同步所有健康历史数据, 会自动根据配置表依次获取支持类型的健康数据; 调用 `syncAllHealthData` 会依次获取健康数据, 并通过 `HealthDataSyncCallback` 回调获取结果.

```
fun removeHealthDataCallBack(syncCallback: HealthDataSyncCallback) 可移除回调;
```

接口说明:

```
DHBleSdk.syncAllHealthData(this)
```

调用示例:

```
DHBleSdk.syncAllHealthData(this)

public interface HealthDataSyncCallback {
    void onSyncProgress(int var1); //同步进度

    void onSyncFinish(); //同步完成

    void onSyncError(int var1);
```

```

void onSyncStep(List<StepSyncBean> var1);

void onSyncSleep(List<SleepSyncBean> var1);

void onSyncHr(List<HeartRateSyncBean> var1); //心率

void onSyncBp(List<BloodPressSyncBean> var1); //血压

void onSyncBo(List<BloodOxySyncBean> var1); //血氧

void onSyncTemp(List<BodyTempSyncBean> var1); //体温

void onSyncPressure(List<PressureSyncBean> var1); //压力

void onSyncBloodSugar(List<BloodSugarSyncBean> var1); //血糖

void onSyncBreath(List<BreatheSyncBean> var1);

void onSyncHrv(List<HrvSyncBean> var1); //HRV

void onSyncMuslimCount(List<MuslimCountSyncBean> var1); //赞念
}

```

3.2.2.3.1 获取单个健康数据内容

⚠ Caution

需设备支持对应健康数据类型; 不支持的将不会同步;

接口说明:

```
fun syncHealthDataByType(type: Int, syncCallback: HealthDataSyncCallback)
```

参数说明:

参数	类型	说明	值
type	Int	整形	见 RingHealthType 几个定义;

调用示例:

```

//获取今天步数数据
DHBLESdk.syncHealthDataByType(Constants.RingHealthType.TODAY_STEP, this)

```

3.2.2.4 全天检测-健康数据说明

1. 今天与历史计步数据 void onSyncStep(List var1)

⚠ Caution

如果有历史数据会分两次回调, 第一次为今天的数据,只会有一天的即一条数据; 第二次为历史计步数据,就可能会返回多天的;

今天与历史计步数据都使用 StepSyncBean, 区别如下:

数据	回调内容	items 内容	总数来源
今天计步	第一次回调，通常只有一个 <code>StepSyncBean</code>	设备当天实际返回的明细	使用设备返回的当天总步数、总卡路里和总里程
历史计步	第二次回调，可能包含多个日期的 <code>StepSyncBean</code>	按日期分组后的设备实际明细	对当天历史明细的步数、卡路里和里程分别求和

```
public class StepSyncBean {
    private long time; //日期时间戳
    private int totalSteps; //总步数
    private int totalCalorie; //总卡路里cal
    private int totalDistance; //总里程m
    private int itemCount; //数据量
    private int activityDataInterval; //计步明细间隔（分钟），默认60
    private List<StepItemBean> items; //设备实际返回的计步明细

    private String date;
    private String hour;
}

public class StepItemBean {
    private long timestamp; //Unix时间戳，单位秒
    private int index; //当天按当前计步粒度划分后的明细序号
    private int steps; //步数
    private int calorie; //卡路里
    private int distance; //里程
}
```

`activityDataInterval` 表示 `items` 的计步明细间隔，单位为分钟；
60 表示每小时一条，10 表示每10分钟一条，未配置时默认为 60。
请使用 `timestamp` 作为明细的准确时间。

2. void onSyncSleep(List var1);

⚠ Caution

返回的为设备里多天所有睡眠状态数据；

```
public class SleepSyncBean {
    private long time; //睡眠当天时间戳 秒(s)
    private long totalSleepTime; //睡眠总时长 分钟(min)
    private long asleepTime; //睡眠开始时间戳
    private long awakeTime; //睡眠结束时间戳
    private int itemCount; //睡眠状态个数
    private List<SleepItemBean> items; //睡眠状态详细值
}

public class SleepItemBean {
    private int len; //当前睡眠类型时长 分钟(min)
    private int sleepType; //睡眠类型：0为清醒 1为浅睡 2深睡
}
```

3. 心率数据 void onSyncHr(List var1)

⚠ Caution

返回多天数据(今天与历史);根据time来区分对应天.

```
public class HeartRateSyncBean {
    private long time; //日期时间戳
    private int itemCount; //数据量

    private List<HeartRateItemBean> items; //数据条目, 对应天的心率值
}

public class HeartRateItemBean {
    private long timeMills; //时间戳
    private int hr;

    private String date;
    private String hour;
}
```

心率变异性(HRV) `HrvSyncBean`, 血氧 `BloodOxySyncBean`, 压力 `BloodPressSyncBean`, 血糖 `BloodSugarSyncBean`, 血压 `BloodPressItemBean` 与心率类似不一一说明

体温 `BodyTempSyncBean`:

```
public class BodyTempSyncBean {
    private long time; //日期时间戳
    private int itemCount; //数据量
    private List<BodyTempItemBean> items;
}

public class BodyTempItemBean {
    private long timeMills; //测量时间戳 秒(s)
    private int temp; //体温原始值(实际温度=temp/10, 如365=36.5℃)
    private String date;
    private String hour;
}
```

4. 赞念数据 void onSyncMuslimCount(List var1)

```
public class MuslimCountSyncBean {
    private long time; //日期时间戳
    private int itemCount; //数据量
    private int totalCount; //总数据量
    private List<MuslimCountItemBean> items;
}

public class MuslimCountItemBean {
    private long timeMills; //测试时间 时间戳 s
    private int count; //计数数量;每小时累加赞念;

    private String date;
    private String hour;
}
```


3.2.3 OTA升级

Note

ota升级文件需从厂家生成取得，确定无误后再进行测试. 防止升级出错变砖.

方法说明:

```
fun ringOtaWithFileData(filePath: String, callback: OnFileTransferCallback)
```

参数说明:

参数	类型	说明
filePath	String	固件文件路径
callback	OnFileTransferCallback	传输进度回调

调用示例:

```
val otaPath = "" //bin文件,厂家提供
DHBleSdk.ringOtaWithFileData(otaPath, object : OnFileTransferCallback {
    override fun onProgress(pro: Float) {
        Log.e("OTA", "progress: $pro")
    }

    override fun onFinish() {
        Log.e("OTA", "OTA finish")
    }

    override fun onFail(code: Int) {
        Log.e("OTA", "OTA fail: $code")
    }
})
```

3.2.4 多运动Workout

Caution

支持多运动配置表属性为 `isNewSport`；开启多运动后, 设备会进入运动中, APP断开与关闭都不会停止,只有通过APP或设备主动停止, 所以带多运动功能的设备,连接后先查询下状态,确定是否在运动中,在多运动状态下会影响其它功能使用;

运动时长需超过2分钟,设备才会保存此次运动数据.

3.2.4.1 获取设备多运动状态

获取设备是否在多运动中; 当前不在运动中时才开启新的运动.

订阅 `SportGetControlCallback` 获取结果.

方法说明:

```
fun controlGetSportJLData()
```

参数说明:

WorkoutControlType枚举	类型	说明	值
Workout_Begin	Int	整形	运动开始
Workout_Continue	Int	整形	运动继续
Workout_Pause	Int	整形	运动暂停
Workout_Finish	Int	整形	运动结束

调用示例:

```
private val sportGetCallback by lazy {
    object : SportGetControlCallback {
        override fun onSuccess() {
            Log.e("RWSDK", "SportGetControlCallback onSuccess")
        }

        override fun onFail(errorCode: Int) {

        }

        override fun onResult(data: NewSportBean) {

            val bleActivityMode = data.sportType
            val tControlType = data.status

            Log.e("RWSDK", "SportGetControlCallback onResult " + bleActivityMode + " " + tControlType)

            //0x01开始 0x03暂停 0x02继续 0x04结束
            if (tControlType?.isInRunning == true) {
                // 在运动中, 直接进入运动

                val intent = Intent(this@WorkoutTypeActivity, WorkoutRunningActivity::class.java).apply {
                    putExtra("bleActivityMode", bleActivityMode?.value)
                    putExtra("controlType", tControlType.value)
                }
                startActivity(intent)
            }
            else{
                if (isItemClick){
                    isItemClick = false

                    DHBleSdk.subscribeData(sportControlCallback)
                    DHBleSdk.controlSportJL(mBleActivityMode!!,
                                            WorkoutControlType.Workout_Begin)

                }
            }
        }
    }
}

DHBleSdk.subscribeData(sportGetCallback)
DHBleSdk.controlGetSportJLData()
```

3.2.4.2 控制设备进入多运动

控制设备进入多运动, 启动运动.

运动中数据变化通过订阅接收 SportDataPushCallback 通知获取.

订阅 SportControlCallback 获取结果.

方法说明:

```
fun controlSportJL(sportType: BleActivityMode, sportStatus: WorkoutControlType)
```

参数说明:

参数	类型	说明	值
sportType	BleActivityMode	运动类型	sportType: 参考BleActivityMode
sportStatus	WorkoutControlType	运动状态	sportStatus: 开始,暂停,继续,结束;参考WorkoutControlType

SportDataPushCallback运动数据变化通知返回数据SportDataPushBean说明:

SportDataPushBean参数	类型	说明	值
ActivityTime	Int	整形	运动持续时间,单位 秒(s);
ActivitySteps	Int	整形	运动中步数
ActivityDistance	Int	整形	运动中产生距离, 单位 米(m);
ActivityCalorie	Int	整形	运动中产生热量, 单位 卡(cal);
ActivityHr	Int	整形	运动中动态心率

调用示例:

```
DHBleSdk.subscribeData(sportControlCallback)
DHBleSdk.controlSportJL(mBleActivityMode!!,
                        WorkoutControlType.Workout_Begin)

private val sportRealPushCallback by lazy {
    object : SportDataPushCallback {
        override fun onSuccess() {
            Log.e("RWSDK", "SportDataPushCallback onSuccess")
        }

        override fun onFail(errorCode: Int) {
            Log.e("RWSDK", "SportDataPushCallback onFail: $errorCode")
        }

        override fun onResult(data: SportDataPushBean) {
            runOnUiThread {
                // 更新运动类型和控制状态
                updateData(data)
            }
        }
    }
}
```

```
}

// 订阅实时运动数据
DHBLESdk.subscribeData(sportRealPushCallback)
```

注意: BleActivityMode 对应名字见示例Demo 字符串strings.xml里定义: `<string-array name="jlrunning_string_array">`

3.2.4.3 控制开启/关闭设备实时通知运动数据

控制开启/关闭设备实时通知运动数据;
运动中数据变化通过订阅接收 `SportDataPushCallback` 通知获取,有时app关闭与进入后台,可告诉设备停止通知数据.

方法说明:

```
fun setExerciseMore(type: Int)
```

参数说明:

参数	类型	说明	值
type	Int	整形	1: 开启通知运动数据; 0: 关闭通知运动数据;

调用示例:

```
//退出运动界面
DHBLESdk.setExerciseMore(0)
```

3.2.4.4 获取多运动数据报告

订阅 `Sport3ResultCallback` 获取结果.

方法说明:

```
fun getSport3ResultJL()
```

返回数据SportResultBean参数说明:

SportResultBean类	类型	说明	值
startTime	long		运动开始时间戳, 单位秒(s)
exerciseTime	long		运动时长,单位秒(s)
workModel	BleActivityMode		运动类型
step	Int		步数,单位步
distance	Int		距离,米(m)
calorie	Int		卡路里, 卡(cal)
viewType	Int		当前运动类型有无步数,步频,配速,距离: 有步频 viewTypeHaveStepFaq: 无步数 viewTypeNoStepNum: 有配速 viewTypeHavePace: 无距离 viewTypeNoDistance:
newSportHrs	数组		当前运动产生的心率列表, 1分钟保存一个;
pacePerKmList	数组		每公里配速列表, 单位秒/公里; 如设备不支持则为null
.....			其它属性见类注释

调用示例:

```
private val sportResult3Callback by lazy {
    object : Sport3ResultCallback {
        override fun onSuccess() {
            Log.e("RWSDK", "Sport3ResultCallback onSuccess")
        }

        override fun onFail(errorCode: Int) {
            Log.e("RWSDK", "Sport3ResultCallback onFail " + errorCode)
            finish()
        }

        override fun onResult(data: List<SportResultBean>) {
            Log.e("RWSDK", "Sport3ResultCallback onResult " + data.size)
            //Save Data
            finish()
        }
    }
}
```

5.2.5 传感器原始数据

PPG/ACC/PPG Red/IR传感器原始数据采集与睡眠实时数据;

配置表属性: isSupportSensorRawPPG (PPG), isSupportSensorRawACC (ACC), isSupportSensorRawPPGRed (PPG Red), isSupportSensorRawIR (IR), isSupportSensorRawSleep (睡眠实时数据);

注意: 睡眠实时数据(sensorType=5)无需手动启动与关闭, 设备支持此功能时会在睡眠过程中自动推送, 通过相同的 `SensorRawDataCallback` 回调接收即可。

sensorType 合法组合:

值	含义	说明
1	ACC	仅ACC
2	绿光(PPG Green)	仅绿光
3	绿光 + ACC	绿光与ACC同时输出
4	红光(PPG Red)	仅红光
5	红光 + ACC	红光与ACC同时输出
10	绿光 + 红外(IR)	绿光与红外同时输出
11	绿光 + ACC + 红外	绿光、ACC与红外同时输出
12	红光 + 红外	红光与红外同时输出
13	红光 + ACC + 红外	红光、ACC与红外同时输出

规则: 绿光与红光不能共存; 红外不能单独启动, 必须与绿光或红光组合使用。

返回数据格式说明:

字段	类型	说明
type	int	数据类型: 1=PPG, 2=ACC, 3=PPG Red, 4=IR, 5=睡眠实时数据
ppgDataList	List<Integer>	PPG数据列表, 每项为int32
accDataList	List<AccRawItem>	ACC数据列表, 每项包含x,y,z (int16)
ppgRedDataList	List<Integer>	PPG Red数据列表, 每项为int32
irDataList	List<Integer>	IR红外数据列表, 每项为int32
sleepDataList	List<long[]>	type=5时的睡眠数据列表, 每项[0]=时间戳(秒), [1]=睡眠模式: 17=睡眠开始, 34=睡眠结束, 1=深睡, 2=浅睡, 3=清醒, 4=REM

AccRawItem 数据说明:

字段	类型	说明
x	int	X轴值 (int16)
y	int	Y轴值 (int16)
z	int	Z轴值 (int16)

5.2.5.0 PPG定时监测

PPG定时监测设置, 类似心率/HRV定时监测;

配置表属性: `isSupportPPGMonitoring`

订阅 `TimedPPGCallback` 获取结果.

方法说明:

```
fun setTimedPPGJL(reminderBean: DrinkReminderBean)
```

```
fun getTimedPPGJL()
```

参数说明:

参数	类型	说明	值
reminderBean	DrinkReminderBean	类	isOpen: true开/false关 remindDuration: 间隔时间默认30分钟 startHour: 0 固定 startMin: 0 固定 endHour: 23 固定 endMin: 59 固定

调用示例:

```
//设置PPG监听
DHBleSdk.subscribeData(ppgDataCallback)
val healthMonitorBean = DrinkReminderBean()
healthMonitorBean.isOpen = true
healthMonitorBean.remindDuration = 60
healthMonitorBean.startHour = 0
healthMonitorBean.startMin = 0
healthMonitorBean.endHour = 23
healthMonitorBean.endMin = 59
DHBleSdk.setTimedPPGJL(healthMonitorBean)

//获取PPG监听
DHBleSdk.subscribeData(ppgDataCallback)
DHBleSdk.getTimedPPGJL()
```

5.2.5.1 启动与关闭传感器原始数据

订阅 `SensorRawControlCallback` 获取启动/关闭结果;

设备也可能主动停止传感器, 此时通过 `SensorRawControlCallback.onResult(reason)` 通知, reason为停止原因(1字节).

方法说明:

```
fun ringControlSensorRaw(outputType: Int, sensorType: Int)
```

参数说明:

参数	类型	说明	值
outputType	Int	输出控制类型	1: 开启Sensor输出 2: 关闭Sensor输出
sensorType	Int	传感器类型(按位组合)	见上方合法组合表

调用示例:

```
//订阅控制回调
```

```

DHBleSdk.subscribeData(sensorRawControlCallback)

//开启PPG+ACC原始数据输出 (sensorType=3)
DHBleSdk.ringControlSensorRaw(1, 3)

//关闭PPG+ACC原始数据输出
DHBleSdk.ringControlSensorRaw(2, 3)

//监听控制结果与设备主动停止
private val sensorRawControlCallback by lazy {
    object : SensorRawControlCallback {
        override fun onResult(data: Int?) {
            Log.e("RWSDK", "device stopped sensor, reason=" + data)
        }
        override fun onFail(errorCode: Int) {}
        override fun onSuccess() {
            Log.e("RWSDK", "sensor control command OK")
        }
    }
}

```

5.2.5.2 数据获取方式

传感器原始数据有两种获取方式, 由设备端决定使用哪种,APP不可选择:

- (1) 实时推送: 启动后设备实时推送数据到APP;
- (2) 历史获取: 设备先采集保存,APP后续主动同步获取;

5.2.5.2.1 实时推送

启动传感器后, 设备实时推送原始数据;

订阅 `SensorRawDataCallback` 获取实时数据; 数据通过 `SensorRawDataBean` 返回.

调用示例:

```

//订阅传感器原始数据回调
DHBleSdk.subscribeData(sensorRawDataCallback)

//开启
DHBleSdk.ringControlSensorRaw(1, 3)

//监听传感器原始数据
private val sensorRawDataCallback by lazy {
    object : SensorRawDataCallback {
        override fun onResult(data: SensorRawDataBean?) {
            data?.let {
                when (it.type) {
                    2 -> Log.e("RWSDK", "ACC count=" + it.accDataList.size)
                    1 -> Log.e("RWSDK", "PPG count=" + it.ppgDataList.size)
                    3 -> Log.e("RWSDK", "PPG Red count=" + it.ppgRedDataList.size)
                    4 -> Log.e("RWSDK", "IR count=" + it.irDataList.size)
                    5 -> Log.e("RWSDK", "Sleep count=" + it.sleepDataList.size)
                }
            }
        }
        override fun onFail(errorCode: Int) {}
        override fun onSuccess() {}
    }
}

```



```
}

```

5.2.5.2.2 历史获取

获取设备保存的传感器历史原始数据, 类似多运动数据同步方式;

订阅 `SensorHistoryRawCallback` 获取数据, `onSuccess` 表示同步完成, `onResult` 返回每包数据.

方法说明:

```
fun ringGetHistorySensorRaw()

```

SensorHistoryRawBean 额外字段:

字段	类型	说明
sequence	int	序号

其余字段(type, timestamp, ppgDataList, accDataList等)与实时推送一致.

`onResult` 返回 `List<SensorHistoryRawBean>`, 包含所有传感器历史记录。

调用示例:

```
DHBleSdk.subscribeData(sensorHistoryRawCallback)
DHBleSdk.ringGetHistorySensorRaw()

private val sensorHistoryRawCallback by lazy {
    object : SensorHistoryRawCallback {
        override fun onResult(data: List<SensorHistoryRawBean>?) {
            data?.let {
                Log.e("RWSDK", "SensorHistory count=${it.size}")
                for (bean in it) { Log.e("RWSDK", " " + bean.toString()) }
            }
        }
        override fun onFail(errorCode: Int) {}
        override fun onSuccess() { Log.e("RWSDK", "SensorHistory sync finished") }
    }
}
```

SDK修订记录

v2.0.0_20260724 (2026.07.24)

- 添加计步明细间隔支持

v2.0.0_20260716 (2026.07.16)

- 优化已知问题

v2.0.0_20260706 (2026.07.06)

- 优化部分数据同步流程的稳定性
- 示例工程优化设备保存与重连流程

v2.0.0_20260616 (2026.06.16)

- 添加计数提醒间隔设置功能(3.2.1.25)
- 功能配置表添加 `isSupportCountReminder`
- 修复历史步数同步items缺少第一条数据(index不从0开始)的问题
- 睡眠数据items新增 `isTemporary` 字段(1:临时数据 0:正式数据)

v2.0.0_20260610 (2026.06.10)

- 添加定时体温监测功能(3.2.2.2.7)
- 功能配置表添加 `isDataTypeTemperature`
- 添加跌落提醒设置功能(3.2.1.24)
- 功能配置表添加 `isSupportFallDetect`
- 修复今天步数同步 `date` 字段为空问题

v2.0.0_20260522 (2026.05.22)

- 添加心率校正功能(3.2.1.23)

v2.0.0_20260507 (2026.05.07)

- 添加睡眠实时数据(`sensorType=5`)支持

v2.0.0_20260505 (2026.05.05)

- 添加震动间隔时长设置(3.2.1.22)

v2.0.0_20260429 (2026.04.29)

- 添加PPG定时监测功能(5.2.5.0)

v2.0.0_20260428 (2026.04.28)

- OTA改成 `ringOtaWithFileData` 接口

v2.0.0_20260414 (2026.04.14)

- 修复获取睡眠模式不正确问题
- 修复设备计数长按清零时回调 `onSuccess` 而非 `onResult` 问题

v2.0.0_20260408 (2026.04.08)

- 添加传感器原始数据历史获取功能(5.2.5.2.2)
- 添加闹钟震动时长设置(3.2.1.20)
- 添加触摸事件通知(3.2.1.21)

v2.0.0_20260327 (2026.03.27)

- 多运动报告数据添加每公里配速(`pacePerKmList`)字段
- 去掉SDK内部Crash捕获动作

v2.0.0_20260314 (2026.03.14)

- 传感器原始数据添加PPG Red(`type=3`)和IR红外(`type=4`)类型
- 功能配置表添加 `isSupportSensorRawPPGRed` 和 `isSupportSensorRawIR`

v2.0.0_20260309 (2026.03.09)

- 传感器原始数据加 `type=0` 时间戳输出

v2.0.0_20260303 (2026.03.03)

- 添加Muslim计数清零方式设置与获取功能

v2.0.0_20260302 (2026.03.02)

- 添加血压监测定时设置与血压数据同步
- 添加Muslim时间显示模式设置
- 添加传感器原始数据(PPG/ACC)获取功能

v2.0.0_20260225 (2026.02.25)

- 添加设置时间格式12/24小时制

v2.0.0_20260208 (2026.02.08)

- 修复 `items` 睡眠详细数据会丢第一个状态问题
- OTA升级加超时处理

v2.0.0_20260130 (2026.01.30)

- 加SDK版本号接口
- 加功能配置表
- 加震动、睡眠模式、亮屏时长、抬腕亮屏、消息推送、心率血氧报警等设置新功能指令
- 加多运动功能指令

blesdk-release-260105 (2026.01.05)

- 设置亮屏时长导致睡眠模式问题

blesdk-release-250827 (2025.08.27)

- Vape修改

blesdk-release-250821 (2025.08.21)

- 赞念可设置测试值
- 睡眠从 `onSyncSleep` 返回
- 启动单次赞念也实时同步数据
- 获取目标与模式时返回模式值

blesdk-release-250811 (2025.08.12)

- 添加Muslim定制接口
- 添加健康数据文档说明

blesdk-release-250723 (2025.07.23)

- 添加Muslim定制产品相关功能

blesdk-release-250418 (2025.04.08)

- 添加戒指产品功能

blesdk-release-241201 (2024.12.01)

- 添加电子烟新功能

联系方式 / 技术支持

- 技术支持邮箱 developer@dhhouse88.com